

Reviewer Pack — Minimal Tensor Product Rank of Tropicalized Boolean Algebras...

Entry 44664115818c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 15:21:10 UTC

Minimal Tensor Product Rank of Tropicalized Boolean Algebras vs ACC⁰ Circuit Threshold

Entry ID: 44664115818c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 15:21:10 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity

Statement:

[‘For every explicit function f in P with an ACC⁰ circuit threshold t , the minimal tensor product rank of the tropicalized Boolean algebra associated with f is $\Theta(t)$.’, ‘This implies that for a given explicit function in P and its ACC⁰ circuit threshold, the tensor product rank of its tropicalized Boolean algebra is linearly proportional to the circuit threshold.’, ‘For all explicit functions in P , there exists a constant c such that the minimal tensor product rank of their tropicalized Boolean algebras is at least c times the ACC⁰ circuit threshold.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a way to represent Boolean algebras in a simplified form, which may expose hidden structures relevant to computational complexity.’, ‘The tensor product rank of a tropicalized Boolean algebra could potentially capture essential properties of a function’s complexity class, such as ACC⁰.’, ‘This conjecture aims to explore the relationship between these two mathematical domains and provide new insights into ACC⁰ lower bounds.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a521b46fa67fff32

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given explicit function in P , if the ratio of the minimal tensor product rank of its tropicalized Boolean algebra to the ACC⁰ circuit threshold is greater than or equal to 0.8 and less than or equal to 1.2 for at least three independent runs.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Tensor Product Rank AND Tropical Geometry AND ACC⁰ Circuit Complexity - Tropicalized Boolean Algebras AND ACC⁰ Circuit Threshold IN Tropical Geometry - Tensor Product Rank $\theta(t)$ IN P AND ACC⁰ Circuit Complexity

Top relevant hits considered: - [http://arxiv.org/abs/1607.03608v3] On the tensor product of linear sites and Grothendieck categories - [http://arxiv.org/abs/1610.00298v4] Khovanskii bases, higher rank valuations and tropical geometry - [http://arxiv.org/abs/2604.11166v2] Asymptotic Behavior of Tropical Rank Functions - [http://arxiv.org/abs/1207.1925v1] Introduction to tropical algebraic geometry - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/math/0403015v1] Amoebas of algebraic varieties and tropical geometry - [http://arxiv.org/abs/2605.07582v1] CP violation analysis of local and nonlocal amplitudes in the $\overline{B}^0 \rightarrow \overline{K}^{*0} \mu^+ \mu^-$ decay - [http://arxiv.org/abs/2508.18081v2] GWTC-4.0: Methods for Identifying and Characterizing Gravitational-wave Transients

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_explicit_function(n):
        # Generate a random polynomial over GF(2) with degree n
        return [random.choice([0, 1]) for _ in range(n + 1)]

    def acc0_circuit_threshold(f):
        # Simulate the ACC0 circuit threshold (simplified)
        return len(f) - 1

    def tropicalized_boolean_algebra(f):
        # Convert polynomial to tropicalized Boolean algebra
        tba = []
        for i in range(len(f)):
            if f[i] == 1:
                tba.append(i)
```

```

    return tba

def tensor_product_rank(tba):
    # Compute the minimal tensor product rank (simplified)
    return len(tba) if tba else 0

n = random.randint(5, 40)
f = generate_explicit_function(n)
threshold = acc0_circuit_threshold(f)
tba = tropicalized_boolean_algebra(f)
rank = tensor_product_rank(tba)

ratio = rank / threshold if threshold != 0 else float('inf')
conjecture_holds = 0.8 <= ratio <= 1.2

return {
    "metric_name": "Ratio of Tensor Product Rank to ACC0 Circuit Threshold",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Function: {f}, Rank: {rank}, Threshold: {t}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_ratio) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ame': 'Ratio of Tensor Product Rank to ACC0 Circuit Threshold', 'metric_value': 0.3, 'instances_tested': 1
TRIAL: {'metric_name': 'Ratio of Tensor Product Rank to ACC0 Circuit Threshold', 'metric_value': 0.666, 'instances_tested': 1
TRIAL: {'metric_name': 'Ratio of Tensor Product Rank to ACC0 Circuit Threshold', 'metric_value': 0.666, 'instances_tested': 1
TRIAL: {'metric_name': 'Ratio of Tensor Product Rank to ACC0 Circuit Threshold', 'metric_value': 0.518, 'instances_tested': 1

```

TRIAL: {'metric_name': 'Ratio of Tensor Product Rank to ACC⁰ Circuit Threshold', 'metric_value': 0.761
 TRIAL: {'metric_name': 'Ratio of Tensor Product Rank to ACC⁰ Circuit Threshold', 'metric_value': 0.388
 TRIAL: {'metric_name': 'Ratio of Tensor Product Rank to ACC⁰ Circuit Threshold', 'metric_value': 0.4,
 TRIAL: {'metric_name': 'Ratio of Tensor Prod

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results show that for at least one instance, the ratio of the minimal tensor product rank to the ACC⁰ circuit threshold is outside the pre-re | next: Investigate functions with a ratio outside the support condition and analyze their properties to understand why they do not conform to the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16795
2	preregistration	ollama_remote	glm4:latest	0	0	9380
3	novelty	ollama_remote	glm4:latest	0	0	8312
4	novelty	ollama_remote	glm4:latest	0	0	11485
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16878
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32918
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9894
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9017
9	judge	ollama_remote	glm4:latest	0	0	54313

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 168992 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/44664115818c.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/44664115818c.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/44664115818c.tar.gz* (if generated)